Strategy

Strategy · Swap cart pricing (percent vs flat vs half) · Build with Waqar

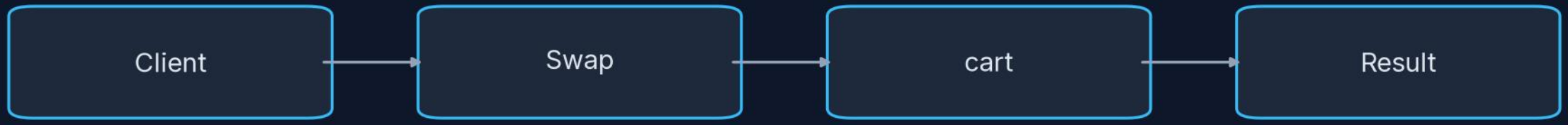
Today's Hook

- Today's force: Swap cart pricing (percent vs flat vs half)
- We earn Strategy by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: Pricing, PercentOff, FlatOff, HalfOff

The Problem

- Scenario: Swap cart pricing (percent vs flat vs half)
- Constraints: solve «Swap cart pricing (percent vs flat vs half)» with Strategy — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Swap cart pricing (percent vs flat vs half)
- Success looks like: 900

Structure



Strategy: Swap cart pricing (percent vs flat vs half)

Real-World Scenario

- Running example: Swap cart pricing (percent vs flat vs half)
- Key types: Pricing, PercentOff, FlatOff, HalfOff
- Happy-path stdout cue: 900
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```java
package com.buildwithwaqar.patterns.gof.behavioral.strategy.problem;

import java.util.*;

public final class NaiveCart {
  public static int total(int cents, String mode){
    if("pct".equals(mode)) return cents*90/100;
    if("flat".equals(mode)) return Math.max(0,cents-200);
    return cents/2;
  }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Swap cart pricing (percent vs flat vs half)
- Without Strategy, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```
package com.buildwithwaqar.patterns.gof.behavioral.strategy.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.function.*;

interface Pricing { int apply(int cents); }
final class PercentOff implements Pricing { private final int p; PercentOff(int p){this.p=p;} public int apply(int c){ return c*(100-p)/100; } }
final class FlatOff implements Pricing { private final int f; FlatOff(int f){this.f=f;} public int apply(int c){ return Math.max(0,c-f); } }
final class HalfOff implements Pricing { public int apply(int c){ return c/2; } }
final class Cart {
  private final Pricing pricing; Cart(Pricing p){ pricing=p; }
  int total(int cents){ return pricing.apply(cents); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.behavioral.strategy.problem;
import java.util.*; import java.util.concurrent.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Strategy: cart pricing ===");
    System.out.println("--- naive ---"); System.out.println(NaiveCart.total(1000,"pct"));
System.out.println("--- strategy ---");
System.out.println(new Cart(new PercentOff(10)).total(1000));
System.out.println(new Cart(new FlatOff(200)).total(1000));
System.out.println(new Cart(new HalfOff()).total(1000));
  }
}
```

Walkthrough Output

```
=== Strategy: cart pricing ===
--- naive ---
900
--- strategy ---
900
800
500
```

Step-by-Step Trace

- Match stdout to stages — look for: 900
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp ... 23-strategy ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
// Java 25 direction for Strategy
// Prefer records + sealed where they shrink the demo.
// Problem under test: Swap cart pricing (percent vs flat vs half)
```

Trade-offs

- Use Strategy when «Swap cart pricing (percent vs flat vs half)» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Strategy in product language.

Related Patterns

- Name the pattern usually confused with Strategy and separate the forces.
- Sketch the roles for «Swap cart pricing (percent vs flat vs half)» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Swap cart pricing (percent vs flat vs half)
- Naive pain → Strategy boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.